

Suning Hermes Agent 项目理解与面试表达

本文以当前工作区代码、数据库迁移与本地模拟数据为第一事实来源，以 README、需求和设计文档为辅助来源。文档中的“已经实现”只表示能在当前代码中找到完整调用链或测试，不等同于已经在生产环境获得业务效果；无法从仓库确认的线上规模、真实数据源、实际 KPI 和部署状态会明确标注。

一、项目一句话定位

Suning Hermes Agent 是一个面向苏宁内部售后运营、客服、质量和管理人员的多 IM 对话式分析系统：员工可以直接在飞书、企业微信或钉钉中，用自然语言查询退换货、订单、商品、物流和退款信息，系统在权限约束下完成跨域取数、分析和结果解释，核心解决“售后数据分散、临时分析依赖人工、单个客诉链路难以快速定位”的问题。

面试时更短的开场可以说：

我做的是一个零售售后领域的内部 Agent。它把原来需要在订单、售后、物流、支付等系统间来回查找的数据，通过带 RBAC 的 MCP 工具统一提供给 IM 中的 Agent，让不同岗位能够用自然语言完成统计分析和单个订单追踪，同时通过上下文、知识检索和长期记忆减少重复沟通。

当前实现边界

- 系统是内部员工的只读查询和分析助手，不是面向消费者的客服机器人。
- 售后业务数据侧只提供查询，不会发起退款、审核退货、修改工单或写回订单系统。
- “跨系统”在当前实现中体现为六个按领域拆分的 MCP 服务和多张业务表；仓库没有展示它们与真实生产订单、WMS、物流、支付系统的独立接口对接，当前本地数据是模拟数据。
- 本次分析实际读取的本地库只有 15 笔订单、15 笔退单、13 条工作流记录、8 个 SKU 等小规模样例，足以验证链路，不足以证明生产数据规模和性能。
- 系统形成了“提问—取数—分析—经验沉淀”的分析闭环，但没有形成“发现异常—自动执行处置—效果自动验证”的执行闭环。

需求/设计文档与当前实现的主要差异

文档中的描述或容易产生的理解	当前代码事实	面试中的准确说法
前端处于 P2 占位或只选了 Vue/Vite	已有 Vue 3、Element Plus、ECharts 管理端，覆盖仪表盘、用户权限、MCP、对话、Skill 五类页面；后端管理 API 也已实现	管理控制台已经可用，但它是运维治理面，不是售后业务操作台
核心 MCP 是五个服务	当前是订单、售后、商品、物流、支付、时间线六个 Streamable HTTP 服务，端口为 8101 ~ 8106	五个数据域服务加一个跨域编排服务
定时日报/周报、Celery、Redis Broker 已形成主动推送	代码中有 Cron 服务身份和售后热点统计缓存，但仓库内没有任务调度定义、日报生成和推送链路，也没有 Celery/MQ 依赖	当前主要是按需查询；只落地了定时预计算所需的身份与缓存基础能力

文档中的描述或容易产生的理解	当前代码事实	面试中的准确说法
已接入真实生产只读库或各业务系统	当前配置指向 MySQL，本地库中订单、退货等核心表各只有十几条模拟记录；查询代码未证明生产副本接入	已完成领域模型、查询服务和安全机制，真实生产数据接入情况无法从当前项目确认
系统会自动监控异常并执行优化动作	当前没有持续异常检测、自动审批、自动退款/调价/拦截和效果验证	系统提供按需分析与决策支持，处置动作仍由业务人员在外部系统完成
使用 LangGraph 编排多 Agent	当前没有 LangGraph；复杂分析使用自定义 DAG、 <code>asyncio</code> 分层并发和 Hermes 的子 Agent 生命周期	使用轻量 DAG 编排，不把框架名称说成业务能力
RAG 已接入完整企业知识库	当前可入库的示例资料只有三份 mock 文本；代码具备切分、向量检索、重排和引用机制	RAG 链路已实现并可扩展，企业级知识覆盖率和线上召回指标无法确认
钉钉只能退化成文字链接	当前代码会尝试上传媒体并发送 Markdown，失败后才退化为文本	三个平台都有适配路径，但具体机器人权限和线上可用性无法从仓库确认
SLA 会按大量品类/业务规则差异化计算	当前有通用阶段阈值，明确的业务特例只有维修且 SKU 为 <code>C1-AC*</code> 时审核响应 2 小时	已实现通用 SLA 状态与一个特例，规则丰富度仍有限
新增 MCP 后会自动发现并注册	Agent 侧暴露的业务工具由插件中的静态 <code>TOOL_SPECS</code> 决定；管理平台注册一条记录不会自动让 Agent 获得新工具	工具运行状态可治理，但工具接入仍需要代码和配置配合
IM 语音是项目内已实现能力	本仓库只消费 Hermes 上游传入的消息上下文，没有语音转写代码	文本消息链路可以确认；语音能力是否由上游 Gateway 提供，无法从当前项目确认
仓库内的所有优化类都已接入主 Agent	<code>inference_optimizer.py</code> 和进程级 <code>agent_harness.py</code> 有实现与单测，但插件注册入口没有调用它们；真正接线的是插件内的会话 Hook、MCP 管理器和复杂分析 Harness	区分“已有实验/基础模块”和“已进入运行主链路的能力”
需求里的时延和效率百分比是项目成果	仓库只给出目标和告警阈值，没有生产指标报表或对照实验	只能作为设计目标，不能在面试中当作已实现 KPI

二、项目背景

1. 所属业务领域

项目属于零售电商的售后服务和经营分析领域。一个商品售出后，如果消费者申请退货、换货或维修，业务会横跨原订单、售后申请与审核、逆向物流、仓库质检、退款支付和商品质量等环节。不同环节由不同岗位负责，也通常落在不同系统或不同数据域中。

这里有两类典型问题：

- **经营分析问题**：例如“近 30 天华东地区空调退货率是否上升”“哪个品牌的质量原因退货更多”。这类问题需要聚合统计、口径约束和维度钻取。
- **个案追踪问题**：例如“某个订单为什么还没有退款”“退货包裹卡在仓库还是支付”。这类问题需要把同一订单下的售后、物流、质检和退款事件按时间串起来。

2. 原来的业务通常如何完成

结合需求文档描述，业务人员通常需要登录多个后台：先在订单系统找到订单，再到售后 workflow 看审核和质检，到物流系统查逆向轨迹，最后到支付系统确认退款。做趋势分析时还要从 BI 或数据库导出 Excel，手动筛选、透视和制作图表。

这种方式并不是“没有系统”，而是已有系统都服务于各自环节，缺少一个面向业务问题的统一查询入口。业务人员真正关心的是“哪里出了问题”和“应该继续找谁”，而不是某一张表或某一个微服务的返回值。

3. 涉及的业务角色

- 售后运营关注整体趋势、退货原因和区域/品类表现。
- 客服主管处理升级客诉，需要快速还原单笔订单全链路。
- 质量工程师从退货原因、SKU、批次和故障现象中发现商品质量线索。
- 区域管理者关注辖区内售后表现，但不应看到无关区域和不必要的敏感信息。
- 高层管理者只需要汇总指标，不应该通过对话下钻到顾客级明细。
- 系统管理员负责人员身份绑定、权限配置、工具启停和运行状况查看。

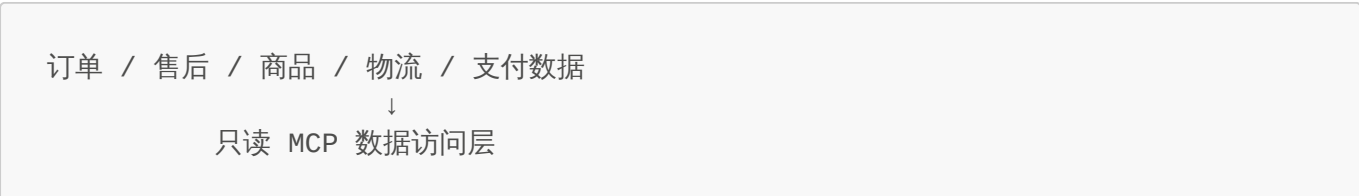
4. 为什么需要建设这个系统

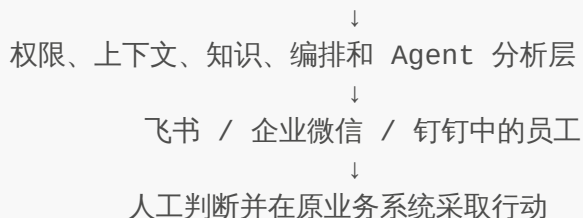
系统建设的重点不是再做一套售后后台，而是给现有数据域增加一个“带业务上下文和权限边界的自然语言分析层”：

1. 用户不需要先知道数据在哪个系统，只要描述业务问题。
2. Agent 可以根据问题选择一个或多个受控工具，而不是让模型直接连接数据库。
3. RBAC 在实际取数前强制生效，避免仅靠提示词限制敏感数据。
4. 多轮上下文、知识检索和历史结论让分析能够连续进行，不必每一轮重新交代条件。
5. 对跨域个案和复杂分析进行专门编排，避免把所有逻辑塞进一次 LLM 调用。

5. 它在业务链条中的位置

它位于“业务数据系统”和“业务人员决策”之间：

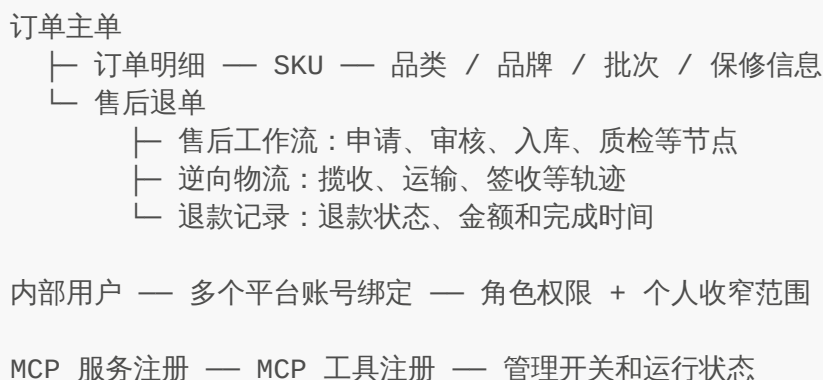




因此，它替代的是查数、拼接、解释和报告准备过程，不替代售后审核、仓库操作、退款执行或管理审批。

6. 核心业务对象及其关系

理解数据库时不需要先背表名，可以先抓住三条业务关系：



订单是交易事实的根，售后退单是一次售后处理的根。workflow、逆向物流和退款都保留退单关联，是为了支持“一个订单多次售后”而不是只按订单号粗暴合并。订单明细单独存在，则是因为 SKU 退货率的分母必须统计“包含该 SKU 的订单”，不能只从退货表反推。

身份、权限和 MCP 注册表与业务表分开，原因是它们属于访问治理元数据，不应混入订单事实。当前六个领域服务最终仍查询同一个 MySQL 实例中的模拟表，所以这里实现的是**逻辑数据域隔离**；是否在生产部署中分别连接真实订单、WMS、物流和支付系统，无法从仓库确认。

三、核心业务痛点

痛点 1：临时经营问题难以及时回答

售后运营遇到临时问题时，需要找 BI 报表、导出数据或请数据人员写 SQL → 固定报表无法覆盖不断变化的品类、区域、原因组合，统计口径还可能因人而异 → 一个本应快速验证的异常线索可能等待数小时甚至更久，业务讨论停留在经验判断。

痛点 2：单个客诉的事实散落在多个环节

客服主管要分别查询订单、售后流程、逆向物流和退款状态 → 各系统使用订单号、售后单号等不同关联键，时间和状态也不在同一个视图 → 定位“到底卡在哪里”耗时，且交接过程中容易漏掉同一订单下的多笔退货。

痛点 3：角色不同，但自然语言入口天然容易越权

不同岗位共用 IM 入口，且同一个人可能绑定多个平台 → 如果只让模型“记住不要泄露”，模型仍可能调用过宽的查询或把上一位用户的信息带入当前会话 → 顾客手机号、人员标识、金额和跨区域明细存在泄露风险，系统也难以满足审计要求。

痛点 4：制度知识与实时业务事实混在一起

业务人员既会问“三包政策是什么”，也会问“这笔订单是否超过处理时限” → 前者来自制度文档，后者必须以当前订单和工作流事实为准，单靠模型常识容易给出过期或没有来源的答案 → 客服解释不一致，甚至会把一般政策错误套用到具体订单。

痛点 5：复杂分析需要人工拆题，任何一步失败都会拖累整份报告

“分析最近 30 天退货趋势、品类、原因、成本并给图表建议”包含多个可并行子问题 → 人工需要多次查数、汇总和排版；单次超长 Agent 调用又容易超时或反复调用相同工具 → 报告交付慢，某个数据源暂时不可用时整项工作可能中断。

痛点 6：历史结论和分析路径没有沉淀

一次分析的过滤条件、结论和有效工具组合通常只存在聊天记录或个人经验中 → 下次遇到相似问题仍要重新查找和组织 → 团队重复消耗查询资源，成熟分析套路也难以复用。

四、核心业务角色

下表结合当前数据库中的五类业务角色权限和管理端能力整理。角色的业务职责部分来自需求语义，能明确确认的是实际数据范围、时间范围和脱敏级别。

角色	负责什么	参与阶段	当前实现中的权限边界
售后运营 aftersale_ops	看整体退换修趋势、原因、区域和品类表现	发起经营分析、根据结果协调业务	不限制区域/品类，最长 365 天，可看完整数据
客服主管 cs_supervisor	处理升级客诉，确认订单卡点与退款进展	个案追踪、向消费者解释、外部协调	华东区域中的南京、上海，最长 30 天，返回脱敏数据
质量工程师 quality_engineer	分析空调、冰箱、洗衣机等品类的退货与故障线索	商品质量分析、供应商或批次排查	仅指定品类，最长 90 天，可看完整数据
区域经理 regional_manager	关注辖区售后表现和区域问题	区域趋势复盘、管理决策	当前样例权限配置为华东及南京、上海、杭州、深圳，最长 90 天，返回脱敏数据
高层管理者 vp_executive	看长期汇总趋势与经营结论	管理复盘和方向判断	最长 365 天，仅允许聚合查询，不能查订单明细
系统管理员	维护身份绑定、权限、MCP 和 Skill 状态，查看运行告警与对话审计	上线前配置、运行期治理和故障排查	使用管理 API；当前是共享 Bearer Token，未实现细分管理员角色

角色	负责什么	参与阶段	当前实现中的权限边界
外部系统/平台	IM Gateway 传递消息， MySQL 提供业务事实，模型 服务提供生成/向量能力	输入、取数、 分析和回复	它们不是业务决策者；真实生 产接入和可用性无法从仓库确 认

一个重要理解是：消费者不是本系统的直接用户。客服人员可能把分析结果用于回复消费者，但系统不会直接受理消费者售后申请。

五、核心业务主流程

1. 业务流程图



2. 当前代码中的实现总图



这里最容易误解的是：仓库实现的重点是 Hermes 的售后领域插件、MCP 服务和管理端，不是从零实现整个 IM Gateway 和通用 Agent Runtime。插件通过 Hook 在上游 Agent 调用模型和工具前后加入身份、上下文、知识和治理能力。

员工查询链路没有再暴露一套要求用户理解字段的订单 REST API，而是向 Agent 暴露以下业务工具；普通 REST API 主要服务于管理控制台：

领域服务	Agent 可调用的实际工具	为什么这样分
订单 8101	search_orders、get_order_detail	区分条件检索与单单详情，避免所有查询都返回大对象
售后 8102	query_return_stats_nl2sql、query_aftersale_nl2sql、query_sku_return_rate、get_aftersale_workflow	同一域内同时保留固定口径、灵活聚合和单据流程三类需求
商品 8103	get_product_info	为售后结论补充 SKU、品牌、保修、故障和批次事实
物流 8104	query_logistics	隔离逆向物流轨迹，允许按退单或订单查询
支付 8105	get_refund_status	隔离退款敏感数据，并只取当前相关退款状态
时间线 8106	trace_order_timeline	在服务端固定并行编排四个领域，避免主模型自行拼错事件
插件本地能力	orchestrate_aftersale_analysis、send_aftersale_chart	前者调度 Hermes 子 Agent，后者需要访问当前 IM 会话并发送本地生成的图片，不属于单一数据域

管理端的 FastAPI 路由集中在 `/api/admin` 下，分别支持用户与平台绑定、MCP 服务/工具、对话审计、Skill 和仪表盘。它们会修改治理元数据，但不会修改订单、退单、物流或退款业务事实。

3. 用业务语言讲完整闭环

系统使用前，管理员先把员工在不同 IM 平台上的外部账号绑定到内部用户，并为该用户分配角色、区域、城市、品类、可查询天数和数据脱敏级别。员工随后可以在日常使用的 IM 中直接提问，不必进入一套新的售后工作台。

消息到达后，系统先确认“是谁在问”，而不是先让模型猜“问了什么”。同一内部员工在私聊场景下可以跨平台延续会话，群聊则保持隔离。系统从当前问题和最近对话中提取主题、日期、区域、品类、品牌、退货原因等结构化条件；如果新问题与上一轮同主题，就继承省略条件，如果主题变化，则清理旧条件，避免把“上次的空调”误套到“这次的冰箱”。

接着，系统根据问题性质选择处理方式。确定口径的问题走固定 SQL 工具；开放但仍属于聚合分析的问题走受限 NL2SQL；追踪单笔订单时并行查询订单、售后、物流和支付并合并成时间线；复杂报告则先拆成趋势、品类、原因、成本等子任务，再分层并行执行。制度、保修等知识问题会补充知识图谱或 RAG 证据，历史相似分析会作为参考，但当前实时业务数据优先。

每一次取数都要经过两层控制：Agent 侧先限制当前场景允许调用什么工具、调用多少次、参数是否合规；MCP 服务侧再校验短期签名、防止令牌重放，并按员工的角色和个人范围注入数据过滤与脱敏。也就是说，即使模型选错工具或构造了过宽条件，最终的数据访问层仍要拒绝或收窄。

结果返回后，Agent 负责把多个结构化结果转成业务可读的解释。某个辅助数据源失败时，系统会明确标注缺失项并尽量返回已有结果，而不是把“查不到数据”和“服务不可用”混为一谈。需要可视化时，可以生成适配桌面和移动端的图表并回复到原会话。

最后，员工根据结果去原业务系统做审核、催办、供应商沟通或退款跟进。本系统不执行这些高风险动作。对话结束后，短期上下文被保存；符合质量阈值的结论和有效分析流程异步沉淀，供后续相似问题召回。因此当前闭环止于“辅助人做决策并积累分析经验”，业务动作及效果验证仍在系统外。

六、核心业务场景拆解

场景 1：按自然语言完成售后经营统计

1. 业务场景

售后运营、区域经理或质量工程师发现某个指标可能异常，例如“最近 30 天华东空调的主要退货原因是什么”，希望快速按日期、地区、品类、品牌或原因组合查询，而不等待新建 BI 报表。

2. 原来的业务问题

固定报表只能回答预设问题；临时问题需要导出 Excel 或请数据人员写 SQL。业务人员会反复修改条件，人工口径容易不一致，也可能直接接触本不该看到的明细数据。

3. 项目的解决方案

员工在 IM 中描述问题，系统提取时间和业务维度，按权限收窄范围后返回聚合结果。常见的趋势、退货原因和 SKU 退货率使用固定查询；只有固定工具无法覆盖的聚合问题才进入 NL2SQL。

4. 核心解决思路

- **固定口径优先**：SKU 退货率的分子是发生退货的去重订单数，分母是包含该 SKU 的去重订单数。这种指标不适合每次交给模型重新解释，因此使用固定参数化 SQL。
- **NL2SQL 只补灵活性**：模型只能在订单、售后、SKU、品类、品牌五张白名单表上生成聚合 **SELECT**，不能生成明细查询、**SELECT ***、任意连接或写操作。
- **权限不是提示词**：区域、城市、品类和最长日期由 MCP 在执行前计算并注入 SQL；用户显式请求越权范围时直接拒绝，而不是让模型自行删改。
- **模拟数据时间处理**：固定统计以业务表中的最大数据时间为参照计算“近 N 天”，这样离线样例不会因为当前日期变化而全部为空。这个处理适合演示数据，真实生产环境是否仍应如此需要重新评估。

5. 技术实现映射

- 售后 MCP 提供固定统计、自由聚合 NL2SQL 和 SKU 退货率工具。
- **nl2sql** 模块完成 Schema 注册、Prompt 版本、候选 SQL 生成、正则与白名单规则校验、RBAC 注入、**EXPLAIN** 和限时执行；当前并未使用完整 SQL AST 解析器。
- MySQL 保存订单、退货、商品及权限数据；Redis 保存成功结果缓存、工具预算和热点预计算结果。
- Tool Governor 控制场景白名单、JSON Schema、同轮去重和最多调用次数。
- 管理端可以启停工具，但新工具仍需要在插件静态工具清单中接入。

6. 最终解决了什么问题

把“找数据人员写 SQL”变成“在权限和统一口径下即时验证业务问题”，同时避免让 LLM 直接拥有任意数据库查询能力。

场景 2：处理升级客诉时追踪一笔订单的全链路

1. 业务场景

客服主管收到消费者催促：“货已经寄回，为什么还没退款？”主管需要用订单号判断申请是否审核、包裹是否揽收和入库、仓库是否质检、退款是否发起，以及当前责任环节。

2. 原来的业务问题

订单、售后、物流、支付各自只展示局部状态。客服需要复制订单号和售后单号在多个系统间切换，人工比时间；同一订单有多笔退货时还容易把不同退货单的事件混在一起。

3. 项目的解决方案

员工只提供一个正整数订单号，时间线服务并行查询订单详情、售后流程、逆向物流和退款状态，把结果转换成统一时间节点，按时间排序后标注每个阶段的耗时、SLA 状态和当前瓶颈。部分来源失败时，仍返回其他已完成节点并说明缺失来源。

4. 核心解决思路

- 顶层令牌只能用于时间线工具，不能直接复用到下游；时间线服务为每个固定下游工具重新签发短期委托令牌，维持工具绑定和一次性 JTI 的安全边界。
- 四个数据源相互独立，因此并行查询比串行更适合客诉场景。整体等待上限为 10 秒，超时任务被取消并记录为 **source_failures**。

- 每个事件保留 `return_id`，避免一个订单多次售后时串单；同一时刻的重复节点按支付、物流、仓库、售后、订单优先级去重。
- SLA 使用申请、审核、揽收、入库、质检、退款等阶段的通用阈值；达到阈值 80% 标记预警，超过阈值标记超时。缺少开始或结束锚点时显示未知，不能伪造耗时。
- 当前瓶颈优先使用数据源明确的待处理阶段；否则根据最后完成节点推导，并明确标记“推导”，不把推理结果冒充源系统事实。

5. 技术实现映射

- 8106 时间线 MCP 负责跨域编排；订单、售后、物流、支付 MCP 提供领域数据。
- `asyncio.wait` 完成并发和整体超时，时间线追踪器负责时间标准化、事件合并、SLA 和瓶颈判断。
- 签名凭证包含用户、平台、会话、工具、签发时间和一次性 JTI；Redis 用 `SET NX` 防重放。
- 顶层 MCP 调用有重试、熔断和降级；但时间线内部四个下游调用当前只使用一次整体超时，没有复用同一套重试/熔断管理器。

6. 最终解决了什么问题

客服不再手工拼接四个数据域，就能快速判断客诉卡点和缺失证据，并把时间用于协调责任环节。

场景 3：查询商品保修政策并结合质量线索判断

1. 业务场景

客服或质量工程师需要回答“某品牌某部件保修多久”“某类故障是否集中在特定 SKU/批次”，既涉及制度文档，也涉及商品、批次和退货事实。

2. 原来的业务问题

政策散落在制度、品牌政策和法规文档中，精确的部件保修期又可能在结构化商品数据里；如果全部依赖全文搜索，容易找不到精确字段，如果全部依赖数据库，又无法解释制度条款和引用来源。

3. 项目的解决方案

系统先用规则识别是否属于政策、保修、故障或批次问题。精确保修关系优先查询 MySQL 中的商品知识关系；一般制度解释使用 Milvus 检索文档片段并重排，回复时要求带来源。商品基础信息仍通过受控 MCP 查询。

4. 核心解决思路

- **结构化事实优先图查询**：品牌、品类、部件和保修时长有明确字段，关系查询比语义相似度更可靠。
- **非结构化规则使用 RAG**：制度条款和说明书语言多样，需要向量召回，再结合关键词、品类、品牌、文档类型和有效期重排。
- **实时事实优先历史知识**：某笔订单是否超时必须看当前工作流；RAG 或历史案例只能补充解释，不能覆盖实时 MCP 事实。
- 当前知识入口用关键词和正则路由，不是由 LLM 自由决定；这样减少每轮额外模型调用，也让路由可解释。

5. 技术实现映射

- 商品 MCP 返回 SKU、品牌、品类、价格、整机/部件保修、常见故障和批次信息。
- Knowledge Hook 在 LLM 前按规则选择 MySQL 知识关系或 Milvus RAG。

- RAG 使用固定长度切块、DashScope Embedding、Milvus 候选召回和业务权重重排，最终最多注入三个来源。
- 当前仓库只有三份 mock 知识文本，无法证明真实企业知识库覆盖率。
- 一个必须指出的安全缺口：知识 Hook 的部分 MySQL 图查询绕过了 MCP/RBAC，特别是故障—批次—退货计数关联可能返回业务聚合信息；因此不能笼统声称“所有业务数据库读取都经过 MCP 权限控制”。

6. 最终解决了什么问题

把“查政策”和“查商品/质量事实”分开走最适合的数据路径，再在回复中合并，减少用模型常识替代企业事实的风险。

场景 4：多轮追问和跨 IM 延续同一分析

1. 业务场景

运营先问“最近 30 天空调退货趋势”，随后只说“华东呢”“再看格力”“主要原因是什么”；或者同一员工从飞书切换到企业微信继续私聊，希望系统知道前文，但群聊内容不能串入私聊。

2. 原来的业务问题

普通无状态接口要求每轮重复所有筛选条件。简单拼接完整聊天记录会不断增加 Token，也会把旧主题、其他群或其他账号的内容错误带入，造成查询条件污染和数据泄露。

3. 项目的解决方案

系统先把不同平台账号映射为统一内部身份。私聊按内部用户复用有一个有效期的逻辑会话，群聊按平台、群和用户隔离。每轮只保存结构化主题、筛选槽位、摘要和少量最近原文；同主题追问合并条件，主题切换清理条件。

4. 核心解决思路

- 省略和指代不是简单正则能稳定解决的，因此用 LLM 提取结构化槽位；但结果必须通过 Pydantic 类型校验和数据库业务值白名单，置信度低于 0.7 时不接受新条件。
- Redis 中的短期上下文保留一小时；超过十轮会压缩较早内容，仅向模型注入压缩摘要和最近三轮原文，控制 Token 增长。
- 同一逻辑账号用五分钟可续租的分布式锁串行处理整轮对话，避免两条并发消息同时修改上下文和工具预算。
- 私聊可以跨平台延续，是因为会话按内部用户而不是外部账号保存；群聊永不恢复私聊上下文，优先保证隔离。
- 长期记忆只记录高置信度的结构化业务结论，不保存完整聊天。SQLite 是事实存储，Milvus 是可重建的语义索引；向量库失败时仍可用全文检索。

5. 技术实现映射

- Identity Hook 从 Hermes 的可信 ContextVar 获取平台身份，查平台绑定和内部用户，模型参数不能自行声明身份。
- Redis 保存活动会话、短期上下文、整轮锁、工具预算和防重放标记。
- DeepSeek 负责主题/槽位和长期记忆抽取；Pydantic、数据库白名单和置信度阈值负责确定性校验。
- 长期记忆使用 SQLite FTS5 与 Milvus 混合召回，综合文本相似度、向量相似度和时间衰减，最终注入少量历史结论。

6. 最终解决了什么问题

让业务人员能够像与同事讨论一样逐步缩小问题，又不以无限堆叠聊天记录或跨场景串话为代价。

场景 5：把复杂分析拆成可并行、可部分交付的任务

1. 业务场景

售后运营需要一份综合分析，例如“分析近 30 天退货趋势、异常品类、主要原因、成本表现，并给出图表和结论”，单个问题包含多个相对独立的取数和解释任务。

2. 原来的业务问题

业务人员需要人工拆分问题并多次查询；如果交给一个 Agent 从头做到尾，调用链过长、容易重复取数，任一工具超时还可能使整份报告失败。

3. 项目的解决方案

主 Agent 可以调用复杂分析工具。规划模型把问题拆成趋势、品类、原因、成本和图表建议等最多五个任务，形成无环依赖图；同一层无依赖任务并行运行，每个查询子 Agent 只选一个最合适的业务工具。最后的聚合 Agent 只接收压缩后的任务摘要，明确展示失败项并生成综合报告。

4. 核心解决思路

- 规划结果必须通过任务类型、数量、ID、依赖和环检测；规划模型失败时使用固定 DAG，保证复杂分析仍有可执行路径。
- 每个任务有独立超时，整体也有 120 秒上限。局部任务失败不取消已经完成的结果，这比一个超长串行调用更符合分析报告的可用性要求。
- 子 Agent 的可用工具仍限定为本插件的受控业务工具，不会因为“多 Agent”绕开 RBAC。
- 聚合只接收最多约 1800 字符的子任务摘要，减少上下文膨胀和不同 Agent 之间的提示污染。
- 图表节点目前只生成建议，并不会自动把图发回 IM；真正发送图表仍需主 Agent 另行调用本地图表工具。
- 图表工具接收模型提供的标签和值并校验长度和有限数值，但没有把这些数值强绑定到前一步 MCP 结果，数据来源一致性目前主要依赖 Agent 提示和调用纪律。

5. 技术实现映射

- 插件本地 `orchestrate_aftersale_analysis` 工具负责规划、DAG 校验、拓扑分层和聚合。
- Hermes 子 Agent 生命周期负责创建、等待、取消和清理；并发使用 `asyncio.gather`，没有 `LangGraph`。
- `send_aftersale_chart` 用 Matplotlib 生成桌面/移动端图，分别适配飞书、企业微信和钉钉回复。
- MCPCallManager 为顶层工具调用提供超时重试、按服务熔断和分级降级；失败结果以结构化不可用信息进入最终报告。

6. 最终解决了什么问题

把一份复杂报告从不可控的长链路变成多个受限、可并行、可部分交付的分析任务，某一维度失败时仍能交付其余结论。

场景 6：管理员控制“谁能查什么”并观察系统是否可用

1. 业务场景

新员工需要开通查询、人员岗位或负责区域发生变化、某个 MCP 工具存在风险需要临时下线，或者运维需要判断近期失败是模型、MCP 还是权限问题。

2. 原来的业务问题

如果身份、权限和工具开关散落在配置文件中，每次调整都要改代码或重启；如果只看聊天结果，也无法区分无数据、权限拒绝、超时和服务故障。

3. 项目的解决方案

管理员通过控制台维护内部用户、平台账号绑定、角色和个性化范围，查看或切换 MCP 工具状态，浏览会话与链路信息，并从仪表盘查看服务分布和告警。

4. 核心解决思路

- 平台账号只负责找到内部身份，最终权限由角色范围与用户个人范围取交集，个人配置只能进一步收窄，不能扩大角色权限。
- 数据级别分为完整、脱敏和仅聚合；群聊最多只能获得脱敏结果。高层角色即使区域不限，也不能查询顾客级明细。
- 工具开关同时写 MySQL 和 Redis，下一次调用前由 Tool Governor 读取，适合紧急止损。
- 可观测性记录 Agent、LLM 和 MCP span、耗时、Token、重试、熔断和降级；管理端从日志和 Hermes 状态库读取展示。
- 管理 API 当前只支持一个共享 Bearer Token；未配置 Token 时仅允许本机回环访问。它没有细粒度管理员角色、审批流和完整的管理操作审计，因此不应表述为成熟 IAM 平台。

5. 技术实现映射

- FastAPI 管理后端提供用户、绑定、MCP、会话、Skill 和仪表盘 API。
- Vue 3 管理端提供五类页面。
- MySQL 保存用户、平台绑定、角色权限和 MCP 注册信息；Redis 承载运行期开关与状态。
- OpenTelemetry 和 JSON 日志提供链路数据；MCP 心跳可主动探测服务。数据库中 **online** 状态只是登记状态，不等于当前进程一定存活。

6. 最终解决了什么问题

把对话式查询从“谁都能问、出错难定位”变成可配置权限、可紧急停用、可追踪调用链的内部系统。

七、业务问题与解决方案对应关系

业务问题	业务场景	项目解决方案	核心实现思路	最终效果
固定报表覆盖不了临时分析，人工 SQL 口径不稳	售后经营统计	固定指标工具与受限 NL2SQL 组合	确定指标固定 SQL；开放问题仅允许白名单聚合 SQL；执行前注入 RBAC	业务可以快速验证临时问题，指标口径和数据范围仍受控

业务问题	业务场景	项目解决方案	核心实现思路	最终效果
客诉事实散落在订单、售后、物流、支付	单笔订单全链路	并行取数并合并统一时间线	委托令牌、并发查询、事件标准化、按退货单隔离、SLA 与瓶颈判断	更快识别卡点，且部分系统失败时保留已有证据
政策文本与精确商品事实混在一起	保修与质量分析	图查询优先精确事实，RAG 补制度解释	规则路由、结构化关系、向量召回、业务重排和来源引用	减少模型凭常识回答政策问题
追问省略条件、跨平台身份割裂	多轮与跨 IM 会话	统一身份、结构化槽位和隔离会话	私聊按内部用户续接，群聊隔离，低置信槽位拒绝，旧内容压缩	可以连续分析，又降低上下文污染和越权风险
综合报告链路长、单点失败影响整体	复杂分析	受限 DAG 拆解、分层并行、局部降级	规划校验、固定回退 DAG、任务/总超时、摘要聚合	并行完成多个维度，允许部分交付
相似问题重复分析	历史经验复用	结构化长期记忆和 Skill 工作流召回	高置信记录、稳定去重键、FTS+向量召回、语义路由提示	复用过去结论和有效步骤，但最终仍由 Agent 重新取实时数据
自然语言入口容易越权和滥用工具	权限与运行治理	身份绑定、MCP 强制 RBAC、工具预算和管理开关	短期签名、防重放、范围求交、脱敏、场景白名单、可观测性	模型无法自行扩大数据权限，故障和拒绝原因可追踪

八、关键业务决策点

决策点	输入是什么	根据什么判断	判断结果	后续流程
是否接受当前用户	Hermes 可信平台、外部账号、会话类型	平台账号能否映射到启用的内部用户；是否为受信 Cron 会话	接受或拒绝	接受后建立逻辑会话；拒绝时不进入 LLM/工具阶段
当前用户实际能查多大范围	角色权限、用户个性范围、请求范围、群聊/私聊	角色与个人区域/城市/品类求交；请求不得超出；日期取更小上限；群聊限制数据级别	完整、脱敏、仅聚合或拒绝	将行过滤和掩码注入查询结果
当前追问是否继承上轮条件	当前文本、旧主题/槽位、LLM 结构化结果	主题是否相同、置信度是否至少 0.7、字段值是否通过类型和业务白名单	合并、清空后重建或保持旧值	形成当前轮确定的业务条件
选择哪种查询路径	问题类型、已提取条件、可用工具	是否是固定口径、开放聚合、单笔跨域或复杂综合问题	固定 SQL、NL2SQL、时间线、复杂编排	进入相应工具或工作流；这一步主要由主 Agent 决定

决策点	输入是什么	根据什么判断	判断结果	后续流程
工具是否允许执行	场景、工具名、参数、当前轮历史、调用次数、管理开关	场景白名单、JSON Schema、同轮去重、预算、缓存和 Redis 开关	命中缓存、允许、拒绝或预算耗尽	允许才签发凭证并调用 MCP
NL2SQL 是否安全可执行	模型生成 SQL、Schema 白名单、角色范围	单语句 SELECT 、仅合法表列/连接、必须聚合、无敏感明细、时间与行数上限、 EXPLAIN 可执行	执行、修复后重试一次或拒绝	成功返回聚合；失败明确为生成/校验/执行问题
知识问题走图查询还是 RAG	用户文本、品牌/品类/部件/故障关键词	精确保修/关联模式优先图查询；一般制度问题走向量检索；图查询无结果可回退 RAG	图结果、文档证据或不注入知识	作为回答证据，但不能覆盖实时 MCP 事实
时间线当前是否异常	各源事件、时间锚点、阶段 SLA、终态	已完成耗时与阈值比较；进行中耗时达到 80% 预警、100% 超时；数据不足为未知	正常、预警、超时、未知、已完成/拒绝	标注瓶颈及其依据，交给客服人员跟进
MCP 失败后是否重试或降级	失败类型、服务熔断状态、工具失败级别	仅明确可重试的超时进行最多三次重试；五次计数失败触发 30 秒熔断；工具分 L1/L2/L3	重试、快速失败、返回降级结果或抛错	L1/L2 尽量保留主回答；合法空结果不会当作故障
复杂分析能否按计划执行	规划模型 JSON、任务类型/数量/依赖	最多五个任务、ID 合法、依赖存在且无环；失败则固定回退计划	接受计划或使用回退 DAG	同层并行执行，失败任务单独记录
是否记录为长期记忆	当前问题、回答、槽位、LLM 抽取结果	主题在允许集合、 should_record 为真、结论非空、置信度至少 0.7	新建、同维度更新或不记录	SQLite 落库，并尽力更新 Milvus 向量索引
是否召回自动沉淀的 Skill	当前查询向量、Skill 触发向量、启用状态、所需工具	余弦相似度至少 0.85，且所需工具属于当前实际可用工具	注入 workflow 提示或忽略	主 Agent 参考步骤执行；Skill 本身不会绕过模型直接执行
是否需要人工审批	查询类型、系统能力边界	当前售后数据工具全部只读，没有退款、审核等业务写操作	查询无需逐次审批；业务动作不在系统内	员工依据结果到原系统执行，当前项目没有审批/回滚链路

有两个判断尤其适合在面试中强调：第一，**模型负责选择与解释，但数据权限由确定性代码裁决**；第二，“不知道”是一种合法结果，时间线缺锚点、数据源失败或知识无证据时，系统不应让模型补造事实。

九、AI / Agent 在项目中的真正作用

1. 普通业务代码负责什么

以下问题不需要 LLM，而且使用确定性代码更可靠：

- 身份绑定、账号启停、角色和个人权限求交。
- 短期签名、工具绑定、防重放、日期/区域/品类限制和字段脱敏。
- 固定指标计算，如 SKU 退货率。
- SQL 白名单校验、**EXPLAIN**、超时和行数限制。
- 时间线事件排序、去重、SLA 阈值判断和终态处理。
- 工具参数 Schema、调用预算、缓存、重试、熔断和降级。
- 管理端 CRUD、服务探测、日志与指标展示。

这些逻辑都具有清晰规则。如果交给 LLM，不但成本更高，还会使权限、指标和故障行为不可重复。

2. 哪些问题更适合使用 LLM

- 把自然语言中的省略、指代和意图转换为结构化主题与筛选槽位。
- 在固定工具无法覆盖时，把聚合业务问题翻译成候选 SQL。
- 判断复杂问题可以拆成哪些分析维度，并组织任务依赖。
- 把多个工具返回的结构化数据综合成业务可读结论。
- 从完成的对话中抽取值得保留的结构化结论和可复用工作流。

这里的共同点是输入表达多样、目标不是单一确定公式，且输出还会经过规则校验或只作为建议，而不是直接改变业务状态。

3. LLM 实际输入了什么业务信息

根据不同阶段，模型可能接收：

- 当前问题和最近三轮原始对话、较早对话的压缩摘要。
- 已校验的主题和槽位，如日期、区域、品类、品牌、原因、SKU、订单号。
- 当前用户允许使用的工具描述，而不是数据库连接。
- MCP 返回的聚合结果、订单事实、时间线节点和结构化失败信息。
- RAG 检索到的少量制度片段、图查询结果和历史相似结论。
- 复杂分析中各子任务的压缩摘要，而不是每个 Agent 的完整上下文。

4. LLM 最终负责什么

LLM 负责“理解、选择、生成候选方案和解释”：识别问题、选择工具、为自由聚合生成候选 SQL、拆解复杂分析、综合结果、生成自然语言回复。它不负责最终身份确认、数据权限裁决、指标公式、SQL 安全放行、SLA 数值判断或任何售后写操作。

5. 为什么不能完全相信 LLM

项目在多个层次增加了确定性约束：

- 槽位必须通过 Pydantic 和数据库业务值白名单，低置信结果不采用。
- 工具参数必须符合 JSON Schema，工具还受场景白名单和次数预算控制。
- NL2SQL 只能访问有限 Schema，且必须通过语句、表列、连接、聚合、敏感字段和时间范围校验。
- MCP 根据服务端解析的身份再次执行 RBAC，模型不能通过参数伪造角色。
- 时间线和 SLA 由代码计算；模型只负责解释计算结果。
- 长期记忆和 Skill 需要达到阈值才保存，召回内容只是辅助提示。

仍然存在的风险也要诚实说明：知识 Hook 的直接图查询未统一进入 MCP/RBAC；图表数值未强绑定到上游查询结果；知识检索失败时当前实现只是停止注入证据，并没有在代码层强制模型拒答。

6. RAG 和历史案例放在哪里

- **企业知识 RAG**：用于售后制度、品牌政策、三包法和说明书类问题。Milvus 负责向量候选，业务规则进行品牌、品类、文档类型、有效期和关键词重排。
- **结构化知识关系**：用于品牌—品类—部件—保修和故障—SKU—批次关联，当前由 MySQL 查询实现，并非 Neo4j。
- **用户长期记忆**：保存某个内部用户过去的结构化结论和条件，SQLite FTS5 是事实库，Milvus 是语义索引。
- **Skill 记忆**：保存被判定为复杂且可复用的工具调用 workflow，通过语义路由向主 Agent 提示。

RAG、长期记忆和 Skill 是三个不同的召回空间：一个回答“公司/品牌规则是什么”，一个回答“这个用户以前得出过什么结论”，一个回答“类似任务以前怎样分析更顺”。

7. 为什么需要 workflow 或 Agent

单个查询可以直接调用 Service，但真实提问往往不能预先确定需要哪个数据域。例如“订单为什么没退款”至少涉及订单是否存在、售后单状态、逆向物流和支付；“做一份退货分析”还需要拆成多个相互独立的统计任务。Agent 的价值在于根据问题动态选择工具，并把异构结果转成一份有上下文的解释。

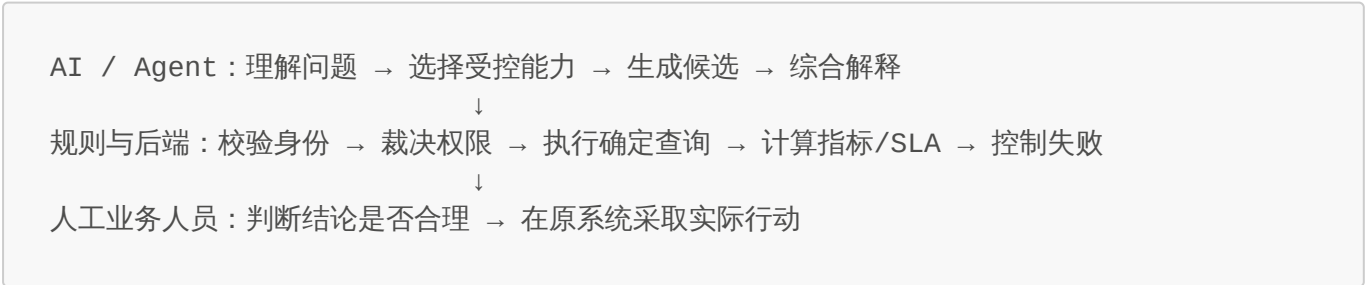
workflow 解决的是 Agent 自由度过高后的工程问题：并发、依赖、超时、局部失败、上下文隔离和结果聚合。当前项目选择自定义轻量 DAG，而不是 LangGraph，因为任务类型和依赖较固定，不需要引入更重的图框架。

8. 人工审批在哪里

当前业务查询全部只读，所以没有“每次查询都审批”的机制。真正高风险的退款、审核、工单修改、回滚等动作没有进入本系统，自然也没有对应审批节点。人工参与发生在 Agent 给出分析之后：业务人员自行判断并在原系统执行处置。

管理员可以人工配置权限、启停 MCP 工具和 Skill，但这属于运行治理，不是售后处置审批。

9. AI 与普通后端如何分工



10. 为什么不是“CRUD 系统加几个 LLM API”

CRUD 适合用户明确知道对象、字段和操作的场景，而本项目的入口是上下文连续、表达不完整、可能跨多个数据域的业务问题。真正的难点不是调用一次模型，而是让模型在可信身份、最小权限、确定指标、工具预算、失败降级和实时数据优先这些约束下完成多步推理。

因此它需要 Agent，但并不意味着所有地方都要 Agent 化：固定统计、RBAC、SLA 和管理操作仍是普通后端代码。项目的合理形态是“LLM 负责不确定性，代码守住确定性边界”，而不是让 LLM 取代 Service 层。

十、项目形成了什么业务闭环

1. 当前真实形成的闭环

业务人员提出问题

- 系统恢复当前上下文和权限
- Agent 选择查询/知识/编排路径
- MCP 返回受控实时事实
- Agent 形成解释或报告
- 业务人员在外部系统处置
- 系统保存本轮上下文、结论和有效工作流
- 下次相似问题召回历史经验并重新查询实时事实

闭环从 IM 中的业务问题开始，以本轮结论和分析路径沉淀结束。下一次提问时，短期上下文帮助继续追问，长期记忆提供过去结论，Skill 提供可复用分析步骤；系统仍会重新调用 MCP 获取当前事实，避免历史结论覆盖数据变化。

2. 反馈如何影响下一轮

- 同一会话的主题和筛选条件直接影响下一轮工具参数。
- 历史结构化结论通过全文和向量召回进入下一次分析背景。
- 复杂且工具路径有复用价值的会话可以生成 Skill，后续相似问题会获得流程提示。
- 工具成功缓存和售后热点预计算减少相同安全范围下的重复查询。
- 调用日志、重试、熔断和管理开关影响后续系统运行策略。

3. 尚未闭合的部分

当前项目没有在代码中实现以下链路：

持续监控经营指标

- 自动发现异常
- 自动创建/审批处置方案
- 写回售后、仓库或支付系统
- 自动验证处置效果

也没有可确认的用户反馈评分驱动 Skill 质量更新。Skill 的 `quality_score` 当前缺少实际反馈闭环，Milvus 长期记忆索引失败后也没有 outbox、重试或自动回填机制。

所以准确说法是：项目形成了**对话分析和经验复用闭环**，没有形成**业务动作和效果验证闭环**。它最区别于普通管理系统的地方，是一次对话可以在权限约束下动态跨域取数，并把上下文、知识和有效分析路径带入下一次决策，而不是多了一个 CRUD 页面。

十一、项目核心价值

1. 减少跨系统查找和手工拼接

客服可以用一个订单号获得订单、售后、物流和退款的统一时间线，不再手工复制多个关联键并比对状态。

2. 更快回答固定报表之外的问题

常用指标使用固定口径，长尾聚合问题使用受限 NL2SQL，使运营能够快速迭代日期、区域、品类和原因，而不让模型拥有任意 SQL 权限。

3. 更早看见处理卡点和证据缺失

时间线对阶段耗时进行正常、预警、超时和未知标记，并区分业务空结果与服务故障，帮助客服知道是“确实没有退款记录”还是“支付数据源暂时不可用”。

4. 降低自然语言查数的权限与稳定性风险

身份来自可信 IM 上下文，MCP 服务端强制行级过滤和脱敏；一次性工具凭证、预算、重试、熔断和降级限制了模型误调用和下游故障的影响范围。

5. 沉淀可复用的分析上下文和方法

结构化长期记忆保存高置信业务结论，Skill 保存有效工具路径。它们不会替代实时取数，但能减少重复澄清和重复设计分析步骤。

不能从当前项目确认的价值包括：真实节省了多少人时、客诉平均处理时长降低多少、退款 SLA 改善多少、线上并发量和模型成本。这些需要生产监控或业务对照数据，面试时不应自行给出百分比。

十二、面试表达版本

1. 30 秒版本

这个项目是面向苏宁内部售后运营、客服和质量人员的对话式分析 Agent。原来他们做退货分析或处理客诉时，需要在订单、售后、物流、支付多个系统之间查找并手工拼数据。我们把这些能力封装成只读 MCP 工具，由 Agent 根据自然语言和多轮上下文选择调用，同时在 MCP 服务端强制做 RBAC、脱敏和 SQL 安全校验。这样既能回答灵活的经营问题，也能生成单笔订单全链路；系统定位是决策支持，不会直接执行退款等高风险动作。

2. 2 分钟版本

项目属于零售售后领域，直接用户不是消费者，而是售后运营、客服主管、质量工程师、区域经理和管理者。业务上有两个主要问题：一类是固定 BI 报表回答不了不断变化的临时问题，另一类是单个客诉的数据散落在订单、售后、物流和支付环节，人工定位非常慢。

我们没有再做一套售后后台，而是把系统放在现有数据域和员工 IM 之间。员工在飞书、企微或钉钉提问后，系统先把平台账号映射为内部身份，恢复当前主题和日期、区域、品类等上下文，再由 Agent 选

择处理方式。固定指标走参数化 SQL，灵活聚合问题走受限 NL2SQL，单笔订单走跨域时间线，综合报告则拆成最多五个子任务并行执行。

项目最重要的设计是把模型能力和业务安全分开。模型负责理解、选工具和解释；MCP 服务端负责验签、防重放、角色与个人范围求交、行级过滤和脱敏。NL2SQL 只能访问五张白名单表，只允许聚合查询，还要经过连接、字段、日期、行数、**EXPLAIN** 和执行超时校验。跨系统调用有重试、熔断和局部降级，某个数据源失败时会明确告诉用户缺了什么，而不是编造结果。

对话结束后，系统会保留短期槽位，并把高置信结论和有效分析 workflow 异步沉淀，下一次相似问题可以复用。不过它目前是只读决策支持系统，业务人员仍要在原系统完成退款、审核或催办；仓库也没有证明生产数据接入和实际 KPI，所以我会把它描述为已完成核心链路的售后 Agent，而不是自动处置平台。

3.5 分钟版本

我做的是一个面向苏宁内部员工的售后分析 Agent。这个领域的特点是，一次售后看起来只是“退货”，实际会穿过订单、售后审核、逆向物流、仓库质检和退款支付。运营关注趋势和原因，客服关注某一单卡在哪里，质量人员关注 SKU、批次和故障，而不同人能看的区域和明细级别还不一样。原有各系统能完成本环节操作，但缺少一个围绕业务问题统一取数和解释的入口。

我们把主流程设计成四层。第一层是身份和上下文。消息从飞书、企微或钉钉进入 Hermes 后，插件从可信上下文读取平台账号，映射到内部用户。私聊可以按内部用户跨平台延续，群聊按群和用户隔离。系统把对话提取成主题、日期、地区、品类、品牌、原因等结构化槽位，同主题追问才继承，置信度低或不在业务白名单的值不会直接进入查询。

第二层是 Agent 路由。常见订单和统计问题走固定工具，因为像 SKU 退货率这样的口径必须稳定；长尾的聚合问题才走 NL2SQL；客诉问题走订单全链路；综合分析会拆成趋势、品类、原因、成本等最多五个 DAG 任务并行执行。我们没有使用 LangGraph，而是用 **asyncio** 和 Hermes 子 Agent 生命周期做轻量编排，因为任务类型和依赖比较固定，没必要增加更重的状态图框架。

第三层是安全数据访问。Agent 侧先用场景白名单、JSON Schema、调用预算、同轮去重和缓存约束工具行为。真正调用 MCP 前，再签发一个只有几十秒有效、绑定用户、会话和具体工具的一次性凭证。MCP 服务验签并用 Redis 防重放，然后把角色范围和个人范围求交，注入区域、城市、品类和日期过滤；脱敏角色会屏蔽手机号、人员标识和金额，高层角色只能调用聚合工具。这样即使模型提示词被绕过，也拿不到更宽的数据。

第四层是结果组织和反馈。举个客诉例子，时间线服务拿到订单号后，会为订单、售后、物流、支付四个下游分别签发委托令牌并并行查询，在十秒内合并完成结果。它按退货单保留事件，计算申请到审核、揽收到入库、质检到退款等阶段 SLA。缺少时间锚点就显示未知；某个来源超时则返回部分时间线和失败来源，不会把系统故障说成没有业务记录。Agent 最后把这些事实解释给客服，实际催办或退款仍由客服去原系统处理。

知识问题采用另一条路径。精确的品牌、部件、保修关系优先查 MySQL 结构化关系，一般政策走 Milvus RAG，再按品牌、品类、有效期和文档类型重排。历史经验也分两类：长期记忆保存某个用户过去的结构化结论，Skill 保存有效分析步骤。它们只作为下一次分析的参考，当前实时 MCP 数据始终优先。

工程上比较有价值的设计，一是把 LLM 的不确定性限制在意图理解、候选生成和结果解释，把权限、指标、SLA 和 SQL 放行留给确定性代码；二是跨域调用有重试、按服务熔断和分级降级；三是复杂报告支持并发和部分交付，不因一个辅助源失败而丢掉所有结果。

我也会主动说明当前边界：仓库使用模拟 MySQL 数据，真实生产系统接入和 KPI 无法确认；定时报表、自动异常发现、业务写回和效果验证没有完整实现；知识库目前只有三份示例文档；知识图谱的部分直接查询还没有统一经过 MCP/RBAC。这说明项目已经形成“提问、取数、分析、经验复用”的闭环，但还不是自动执行售后动作的平台。它的核心价值是减少跨系统查找和重复分析，同时把自然语言查数控制在可审计、可降级的边界内。

面试表达时最容易说错的五件事

1. 不要说“Agent 自动完成退款或售后处置”，应说“给出事实和判断依据，员工在原系统处置”。
2. 不要说“全部查询都走 NL2SQL”，应说“固定指标优先固定 SQL，NL2SQL 只处理受限聚合长尾问题”。
3. 不要说“用了 LangGraph 和消息队列”，当前复杂编排是自定义 DAG + `asyncio`，后台沉淀用线程池/异步任务。
4. 不要说“所有数据库访问都经过 MCP”，知识 Hook 的部分图查询是当前需要补齐 RBAC 的例外。
5. 不要把需求里的时延、效率和主动报表目标说成已验证成果；没有生产数据就明确说无法确认。

十三、项目理解自测

以下问题不附答案，建议先口头回答，再回到代码验证。

1. 为什么 SKU 退货率使用固定 SQL，而自由售后分析才使用 NL2SQL？如果全部交给 LLM，会出现哪些口径和安全问题？
2. 一个飞书用户发起查询后，平台账号如何变成内部用户，角色权限和个人权限如何组合？为什么个人权限只能收窄？
3. 为什么权限必须在 MCP 服务端再次执行，而不能只依赖系统提示词和 Agent 的工具描述？
4. `full`、`anonymized` 和 `aggregated` 三种数据级别分别解决什么业务风险？为什么群聊还要额外收窄？
5. 用户说“再看华东的”时，系统如何判断继承上一轮品类和日期？如果置信度不足或话题改变会发生什么？
6. 同一员工从飞书切到企业微信后，什么情况下可以继续原私聊？为什么群聊不能复用私聊上下文？
7. 时间线为什么需要为每个下游工具重新签发凭证？直接转发顶层凭证会破坏哪些安全约束？
8. 同一订单存在多笔退货时，时间线如何避免把事件混在一起？为什么总 SLA 可能显示未知？
9. 如何区分“没有退款记录”和“支付服务查询失败”？这种区分会怎样影响客服下一步行动？
10. 当前 SLA 判断用了哪些确定性规则？哪些瓶颈是源系统事实，哪些只是系统推导？
11. NL2SQL 从用户问题到 SQL 执行经历哪些校验？为什么 `SELECT` 并不天然等于安全？
12. 复杂分析的计划如果包含不存在的依赖或形成环会怎样？为什么还需要固定回退 DAG？
13. 为什么复杂分析的子 Agent 只拿有限工具和压缩摘要？如果共享完整上下文和全部工具，会增加哪些风险？
14. 知识图谱查询与 RAG 分别适合回答什么问题？为什么精确保修关系不应该只靠向量相似度？
15. 企业知识、用户长期记忆和 Skill 三类内容在数据来源、隔离范围和使用目的上有什么区别？
16. SQLite 长期记忆库与 Milvus 向量索引为什么不是双主存储？如果 Milvus 写入失败，当前系统能保留什么、会失去什么？
17. 自动沉淀的 Skill 是如何被召回的？为什么说它只是给主 Agent 的流程提示，而不是一个可直接执行的确定工作流？
18. MCP 的重试、熔断和降级分别解决什么问题？为什么合法空结果不能计入熔断失败？
19. 当前项目为什么只能称为“分析与经验复用闭环”，不能称为“自动经营优化闭环”？要补齐后者还缺哪些系统能力？

20. 如果要把项目真正接入生产，你会优先补齐知识图谱 RBAC、图表数据溯源、向量索引可靠性、管理端权限、真实数据连接中的哪几项？依据是什么？